

James Hyle

CS450: Programming Language Translation

Dr. Sherman

2/9/26

Milestone 1: ANTLR Project

What didn't work:

On my first try, I attempted to implement the larger pieces first, items such as the do while loop, for loop, and inc-dec statements. This led to many more problems than I anticipated because these statements rely on other pieces of the grammar to be working so that they can function properly. For example, the for loop relies on forInits, and forUpdates which requires both the assignStatement and inc-decStatements to be completed. Stopping progress to go work on another piece of the grammar made my work unorganized and inefficient, and it is hard to hold all the pieces in my head at once. This top-down approach was not the smart way to go; I broke more things than implemented, and as a result, I saved a few things I had been working on and decided to start over on the grammar. It was still a valuable experience to see how the tools worked, and reformed my thinking on how I should tackle the pieces.

What did work:

After restarting my efforts, I decided a much more optimal (and easy) approach was to tackle the smaller pieces and then work my way up to the full grammar specifications. I started with the foundations; unary and binary expressions, type, and assignment were relatively simple to implement. After getting those working, I decided to tackle the assignment and statement pieces. This worked well and I was able to get many tests passing in one session, at that point I believe I was up to ~100 tests passing. I felt comfortable implementing the statement and class/field parts at this point, and got quite a few more tests passing. I do recall that after getting the program/compilation

unit grammar working, I was making strong progress both on the test suite and my theoretical understanding of the grammar. At this point I went on to implement the statements now that many of the building blocks for those pieces were finished. The do-while loop and inc-dec statements went smoothly, however I spent nearly a third of my time working on the for loops; I was running into weird parsing issues with the semicolons in between statements. I used the ANTLR parse tree visual generously to help me understand how the input was being parsed and organized. I have all of the test cases passing, but I feel I could still work on my for loop implementation. I will have to see how the grammar evolves as we continue on to more milestones.

What I learned:

In many ways I felt more comfortable working on implementing the grammar for ESJ than many of the examples worked on in class, I think that is because I am more familiar with Java/programming language syntax. For example, I really struggled with implementing the palindrome grammar in class as it dealt more strictly with regular expressions that I do not have as much experience with. After working with regular expressions in generating Java syntax constructs, I feel like I have a much better handle on them. At this point I am comfortable with ANTLR and using the different tools it provides for viewing the syntax tree, and hierarchy generated in its preview tab. One of the more powerful features I took advantage of when building out the grammar is being able to pinpoint in the input which token to focus on, and reveal its parent and children tokens. This was extremely helpful in diagnosing individual productions and working them through to a valid rule. I am pretty comfortable conceptually with syntax's, but viewing the concrete tree built out in ANTLR helped reinforce how the grammar is put together. I am looking forward to the next milestone, and the next set of rules we are to implement.

I have some experience with test driven development, but this was a really good exercise in utilizing the tests as a mental roadmap of where to head next. Using the tests to break the project into manageable chunks took a lot of pressure off, as it was a bit overwhelming having 138 tests to work through initially. As I said in the what worked section, it was much easier to start with the smaller test cases, and then make my way up to the larger tests. (SyntaxTorture and For were a pain!) Taking a bottom-up approach is definitely the strategy I will be using in future milestones on this project.